

- Description

This module manages a smart streetlight brightness adjustment and energy-saving system, ensuring that:

1. **Streetlight brightness is dynamically adjusted based on ambient light levels and pedestrian activity.**
2. **If ambient light is sufficient, brightness is reduced to save energy.**
3. **If pedestrian or vehicle activity is detected in low-light conditions, brightness is increased for safety.**
4. **A log of all brightness adjustments, energy consumption, and motion detection events is maintained for optimization.**

This system helps improve energy efficiency, enhance public safety, and optimize electricity usage in urban environments.

- Specification

module SmartStreetlightControl

var currentBrightness, maxBrightness, minBrightness, ambientLight: real

var motionDetected: boolean

var log: sequence of string

process updateAmbientLight(newLightLevel: real)

functional scenario form:

ambientLight > 500 && Brightness reduced for energy saving.

||

ambientLight <= 500 and motionDetected=0 && Brightness increased for safety.

||

ambientLight <= 500 and motionDetected=1 && Brightness set to medium level.

end process

process detectMotion(isMotionDetected: boolean)

functional scenario form:

(motionDetected) && Brightness increased for pedestrian safety.

OR

(¬motionDetected) && Brightness adjusted for energy efficiency.

end process

end module

- Java Code

```
package code;
import java.util.*;
class SmartStreetlight {
    private double maxBrightness;
    private double minBrightness;
```

```

    private double currentBrightness;
    private boolean motionDetected;
    private double ambientLight;
    private List<String> log;
    public SmartStreetlight(double maxBrightness, double minBrightness,
double initialBrightness, double ambientLight) {
        this.maxBrightness = maxBrightness;
        this.minBrightness = minBrightness;
        this.currentBrightness = initialBrightness;
        this.ambientLight = ambientLight;
        this.motionDetected = false;
        this.log = new ArrayList<>();
    }
    public void updateAmbientLight(double newLightLevel) {
        ambientLight = newLightLevel;
        adjustBrightness();
    }
    public void detectMotion(boolean isMotionDetected) {
        motionDetected = isMotionDetected;
        adjustBrightness();
    }
    private void adjustBrightness() {
        if (ambientLight > 500) {
            currentBrightness = minBrightness;
        } else if (motionDetected) {
            currentBrightness = maxBrightness;
        } else {
            currentBrightness = minBrightness * 2;
        }
        String message = String.format("Brightness Adjusted: %.2f%% | A
mbient Light: %.2f lux | Motion Detected: %b",
            currentBrightness, ambientLight
, motionDetected);
        System.out.println(message);
        log.add(message);
    }
    public void printLog() {
        System.out.println("\nStreetlight Adjustment Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}
public class c48_SmartCityLightingSystem {

```

```

    public static void main(String[] args) {
        SmartStreetlight streetlight = new SmartStreetlight(100.0, 10.0
, 50.0, 200.0);
        // testcase-VT:
//      streetlight.updateAmbientLight(600.0);
//      streetlight.updateAmbientLight(100.0);
//      streetlight.detectMotion(true);
//      streetlight.detectMotion(false);
//      streetlight.updateAmbientLight(700.0);
//      streetlight.printLog();

        // testcase-FT:
//      streetlight.updateAmbientLight(7.521898124600501E300);
//      streetlight.detectMotion(false);
//      streetlight.updateAmbientLight(0.0);
//      streetlight.detectMotion(false);
//      streetlight.updateAmbientLight(-1.7976931348623157E308);
//      streetlight.printLog();

        // testcase-Z3:
//      streetlight.updateAmbientLight(600);
//      streetlight.updateAmbientLight(100);
//      streetlight.detectMotion(true);
//      streetlight.detectMotion(false);
//      streetlight.updateAmbientLight(700);
//      streetlight.printLog();
        // testcase-UVT:
        streetlight.updateAmbientLight(600.0);
        streetlight.updateAmbientLight(100.0);
        streetlight.detectMotion(true);
        streetlight.detectMotion(false);
        streetlight.updateAmbientLight(700.0);
        streetlight.printLog();
    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c48_SmartCityLightingSystem.java	100.0 %	6	0	6
> c48_SmartCityLightingSystem		0	0	0
> SmartStreetlight	100.0 %	6	0	6

3. Vibration testing instruction coverage: 97.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	136	21,307	21,443
✓ src	0.6 %	136	21,307	21,443
✓ code	0.6 %	136	21,307	21,443
✓ c48_SmartCityLightingSystem.java	97.8 %	136	3	139
> c48_SmartCityLightingSystem	89.7 %	26	3	29
> SmartStreetlight	100.0 %	110	0	110

4. Fuzz testing branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.6 %	5	837	842
✓ src	0.6 %	5	837	842
✓ code	0.6 %	5	837	842
✓ c48_SmartCityLightingSystem.java	83.3 %	5	1	6
> c48_SmartCityLightingSystem		0	0	0
> SmartStreetlight	83.3 %	5	1	6

5. Fuzz testing instruction coverage: 94.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	131	21,312	21,443
✓ src	0.6 %	131	21,312	21,443
✓ code	0.6 %	131	21,312	21,443
✓ c48_SmartCityLightingSystem.java	94.2 %	131	8	139
> c48_SmartCityLightingSystem	89.7 %	26	3	29
> SmartStreetlight	95.5 %	105	5	110

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c48_SmartCityLightingSystem.java	100.0 %	6	0	6
> c48_SmartCityLightingSystem		0	0	0
> SmartStreetlight	100.0 %	6	0	6

7. Z3 instruction coverage: 97.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	136	21,307	21,443
✓ src	0.6 %	136	21,307	21,443
✓ code	0.6 %	136	21,307	21,443
✓ c48_SmartCityLightingSystem.java	97.8 %	136	3	139
> c48_SmartCityLightingSystem	89.7 %	26	3	29
> SmartStreetlight	100.0 %	110	0	110

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c48_SmartCityLightingSystem.java	100.0 %	6	0	6
> c48_SmartCityLightingSystem		0	0	0
> SmartStreetlight	100.0 %	6	0	6

9. Unimproved V-method instruction coverage: 97.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	136	21,307	21,443
✓ src	0.6 %	136	21,307	21,443
✓ code	0.6 %	136	21,307	21,443
✓ c48_SmartCityLightingSystem.java	97.8 %	136	3	139
> c48_SmartCityLightingSystem	89.7 %	26	3	29
> SmartStreetlight	100.0 %	110	0	110